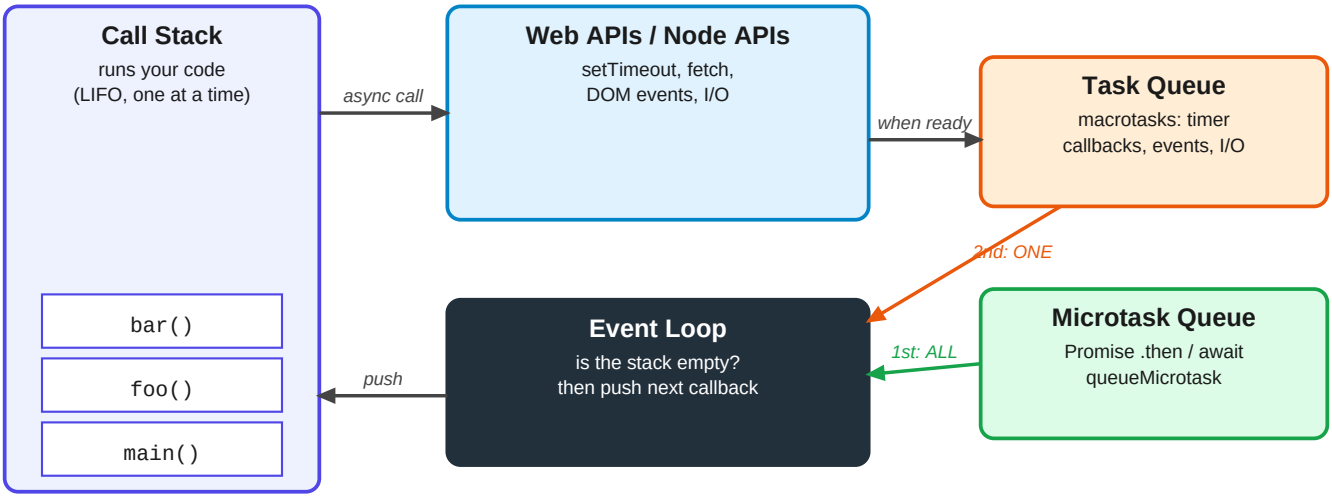
JavaScript Event Loop

Interview notes with examples

The idea in one line: JavaScript is **single-threaded** (one call stack, one thing at a time). The **event loop** lets it handle timers, network and clicks without blocking, by deciding what runs next once the stack is empty.



Part	What it does
Call Stack	Where code executes, last-in first-out. Synchronous code runs here; a long task here blocks everything.
Web APIs (Node: C++ APIs)	Provided by the runtime, not the JS engine: timers, fetch, DOM events. They wait in the background and do not block the stack.
Task Queue (macrotasks, callback queue)	Callbacks from `setTimeout`, `setInterval`, DOM events, I/O. The loop takes **one** per turn.
Microtask Queue	Promise `.then/.catch/.finally`, `await` continuations, `queueMicrotask`, MutationObserver. Drained **completely** before the next task.
Event Loop	Keeps checking: if the stack is empty, move the next callback onto it.

The rule (memorise this)

1. Run the script (all synchronous code) until the call stack is empty.
2. Run **ALL** microtasks - including new ones added while draining.
3. (Browser may render / repaint.)
4. Take **ONE** task from the task queue and run it.
5. Go back to step 2.

> **REMEMBER: Sync code -> all microtasks -> one macrotask -> repeat.** That is why `Promise.then` always beats `setTimeout(fn, 0)`.

Example 1 - the classic

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

OUTPUT: `1 4 3 2`

Step-by-step trace:

Step	What runs	Microtask Q	Task Q	Console
1	`log("1")`	-	-	1
2	`setTimeout(...)` handed to Web API, then callback queued	-	[cb2]	
3	`Promise.then(...)` queued	[cb3]	[cb2]	
4	`log("4")`	[cb3]	[cb2]	4
5	Stack empty: drain microtasks, run cb3	[ ]	[cb2]	3
6	Then take one task, run cb2	[ ]	[ ]	2

Example 2 - microtask that schedules a timer

```
console.log("start");

setTimeout(() => console.log("timeout 1"), 0);

Promise.resolve()
  .then(() => {
    console.log("promise 1");
    setTimeout(() => console.log("timeout 2"), 0);
  })
  .then(() => console.log("promise 2"));

setTimeout(() => console.log("timeout 3"), 0);

console.log("end");
```

OUTPUT: `start end promise 1 promise 2 timeout 1 timeout 3 timeout 2`

- Sync code prints **start**, **end**. Queues now: Microtasks [promise 1], Tasks [timeout 1, timeout 3].
- Drain microtasks: **promise 1** runs and adds timeout 2 to the *end* of the task queue; its chained `.then` is a new microtask, so **promise 2** runs in the same drain.
- Now tasks run in order: **timeout 1**, **timeout 3**, **timeout 2**.

Example 3 - async / await

```
async function a() {
  console.log("a1");
  await b();                 // code after await = a microtask (like .then)
  console.log("a2");
}
async function b() { console.log("b"); }

console.log("script start");
setTimeout(() => console.log("timeout"), 0);

a();

new Promise((resolve) => {
  console.log("p1");         // the executor runs SYNCHRONOUSLY
  resolve();
}).then(() => console.log("p2"));

console.log("script end");
```

OUTPUT: `script start a1 b p1 script end a2 p2 timeout`

- Everything up to the first `await` runs synchronously: **script start**, **a1**, **b**.
- `await b()` pauses `a()`; its continuation (**a2**) goes to the microtask queue and the script keeps going.
- The Promise executor is synchronous: **p1** prints now; `.then` queues **p2**. Then **script end**.
- Microtasks in queue order: **a2**, **p2**. Last, the task: **timeout**.

Example 4 - setTimeout(fn, 0) is not instant

```
const start = Date.now();
setTimeout(() => console.log("timer ran after", Date.now() - start, "ms"), 0);

while (Date.now() - start < 500) {}   // blocks the call stack for 500 ms
```

OUTPUT: `timer ran after ~500 ms`

The delay is a **minimum**. The callback cannot run until the stack is empty, so one slow synchronous task delays every timer, click and repaint.

Common gotchas

- **Promise executor is synchronous;** only `.then/.catch/.finally` callbacks are async.
- **Microtask starvation:** a microtask that keeps scheduling new microtasks never lets a task or a repaint run, and the page freezes.
- **Same delay = scheduling order:** two `setTimeout(fn, 0)` calls run in the order they were created.
- **Node.js:** `process.nextTick` runs before Promise microtasks; `setImmediate` runs in the check phase (after I/O). From the main module, `setTimeout(0)` vs `setImmediate` order is not guaranteed.
- **Not on the loop:** `requestAnimationFrame` callbacks run right before the next paint, after microtasks.

Interview questions - short answers

1. Why is JavaScript non-blocking if it is single-threaded?

Slow work (timers, network, I/O) is handed to the runtime's Web/Node APIs. When it finishes, its callback is queued, and the event loop runs it when the stack is empty.

2. Microtask vs macrotask?

Microtasks (Promises, queueMicrotask) run immediately after the current code and are fully drained before anything else. Macrotasks (timers, events, I/O) run one per loop turn.

3. Why does Promise.then run before setTimeout(fn, 0)?

After the sync code finishes, the loop empties the microtask queue first; only then does it take a task.

4. Is setTimeout(fn, 1000) exactly 1 second?

No. It is the minimum delay before the callback is queued; it runs once the stack is free.

5. How do you avoid blocking the event loop?

Split heavy work into chunks (setTimeout / requestIdleCallback), move it to a Web Worker, or use async APIs instead of long synchronous loops.

Practice - predict the output

Trace with three columns: Stack / Microtasks / Tasks.

```
// Q1
setTimeout(() => console.log("A"), 0);
Promise.resolve().then(() => console.log("B"));
(async () => { console.log("C"); })();
console.log("D");

// Q2
Promise.resolve().then(() => console.log(1));
queueMicrotask(() => console.log(2));
setTimeout(() => console.log(3), 0);
console.log(4);

// Q3
console.log(1);
setTimeout(() => console.log(2), 100);
setTimeout(() => console.log(3), 0);
Promise.resolve().then(() => console.log(4));
console.log(5);

// Q4
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => {
  console.log("micro 1");
  Promise.resolve().then(() => console.log("micro 2"));
});

// Q5
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);
```

Q	Output	Why
1	C D B A	The async function body is sync until await, so C prints first; then D; microtask B; then task A.
2	4 1 2 3	Sync 4; microtasks 1 and 2 in the order queued; then the timer 3.
3	1 5 4 3 2	Sync 1, 5; microtask 4; timers fire by delay: 0 ms (3) before 100 ms (2).
4	micro 1 micro 2 timeout	A microtask added during draining also runs before the next task.
5	3 3 3	One shared `var i` is already 3 when callbacks run. Use `let` to get 0 1 2.